

Hadoop MapReduce Client Core

核心架构设计文档

分支: release-3.3.5-RC0

模块: hadoop-mapreduce-client-core

源文件总数: 537 个 Java 文件

Old API (mapred): 205 文件 | New API (mapreduce): 332 文件

自动生成 · 基于源码深度分析

目录

1. 项目概述与模块定位
2. 分层架构总览
3. 核心包结构与文件统计
4. 核心类关系分析
5. 设计模式分析
6. 核心流程一：作业提交流程
7. 核心流程二：MapTask执行流程
8. 核心流程三：ReduceTask与Shuffle流程
9. 核心流程四：输出提交协议
10. 时序图：Job提交与执行
11. 时序图：Shuffle数据拉取
12. 关键场景调用链分析
13. 配置参数速查
14. 总结

1. 项目概述与模块定位

hadoop-mapreduce-client-core 是 Hadoop MapReduce 框架的客户端核心库，为 MapReduce 编程模型提供完整的客户端 API、作业提交机制、任务执行引擎、Shuffle 数据传输以及输出提交协议的实现。该模块是连接用户代码与集群运行时环境的桥梁。

核心职责：定义 MapReduce 编程 API (Mapper/Reducer/InputFormat/OutputFormat) → 封装作业提交管道 (Job/JobSubmitter/Cluster) → 实现 Map/Reduce 任务执行引擎 (MapTask/ReduceTask) → 实现 Shuffle 数据传输 (Fetcher/MergeManager/Merger) → 管理输出提交协议 (OutputCommitter/FileOutputCommitter)

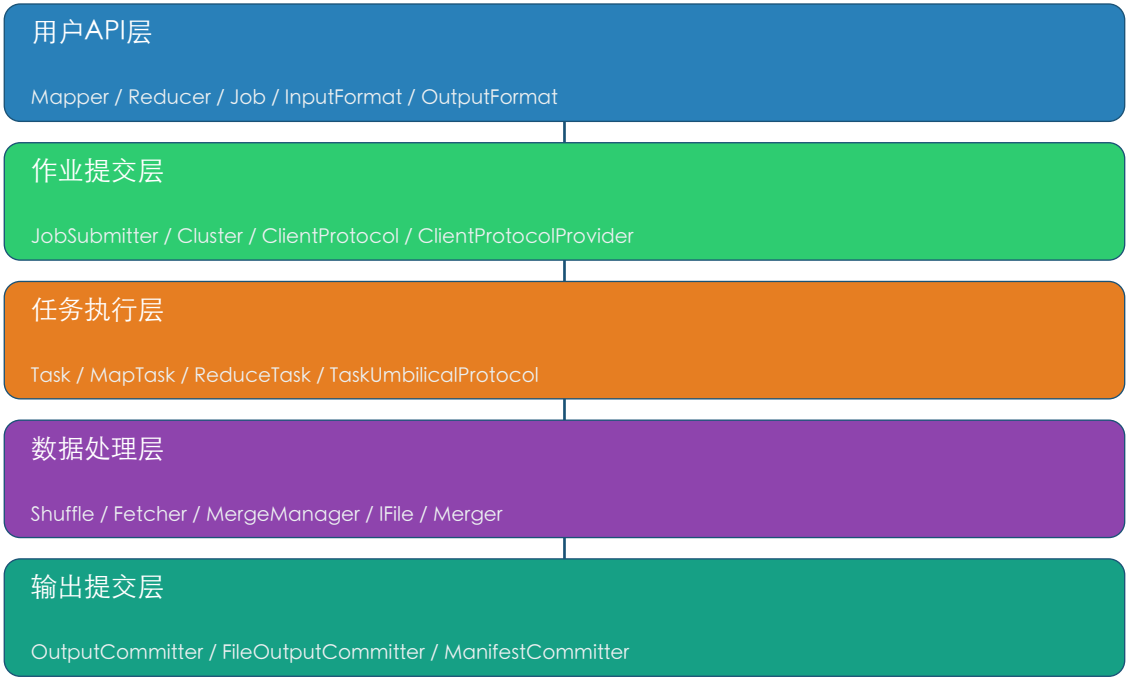
1.1 双API架构

该模块维护了两套并行的 API 体系，这是理解整个代码库的关键前提：

API版本	包名	文件数	设计特点	状态
Old API (MR1)	org.apache.hadoop.mapred	205	基于接口+JobConf配置, 回调式	兼容维护
New API (MR2)	org.apache.hadoop.mapreduce	332	基于抽象类+Context对象, 面向对象	推荐使用

2. 分层架构总览

从功能职责维度，整个模块可清晰划分为五层架构，每层有明确的职责边界：



2.1 各层职责详解

层次	核心类	职责描述
用户API层	Mapper, Reducer, Job, InputFormat, OutputFormat, Partitioner, RecordReader/Writer	提供面向用户的MapReduce编程抽象，用户只需继承/实现这些类即可编写MR程序
作业提交层	JobSubmitter, Cluster, ClientProtocol, ClientProtocolProvider (SPI)	封装作业提交管道：检查规范→拷贝资源→切分输入→写配置→通过RPC提交到集群
任务执行层	Task, MapTask, ReduceTask, TaskUmbilicalProtocol	在TaskTracker/NodeManager上执行Map/Reduce任务，管理任务生命周期和进度汇报
数据处理层	Shuffle, Fetcher, ShuffleScheduler, MergeManagerImpl, IFile, Merger	实现Reduce端Shuffle：拉取Map输出→内存/磁盘多阶段合并→生成有序迭代器
输出提交层	OutputCommitter, FileOutputCommitter, PathOutputCommitter, ManifestCommitter	两阶段提交协议：task commit + job commit，保证输出原子性，支持推测执行和故障恢复

3. 核心包结构与文件统计

3.1 Old API 包结构 (org.apache.hadoop.mapred, 205文件)

子包/核心类	文件数	功能描述
mapred/ (根)	~60	核心任务执行: Task, MapTask, ReduceTask, JobConf, IFile, Merger, Counters
mapred/lib/	~30	内置InputFormat/OutputFormat实现: KeyValueText, NLine, Chain, MultipleOutputs
mapred/join/	~15	多源连接输入格式: CompositeInputFormat, JoinRecordReader
mapred/pipes/	~5	C++ Pipes接口支持

3.2 New API 包结构 (org.apache.hadoop.mapreduce, 332文件)

子包	文件数	功能描述
mapreduce/ (根)	~40	核心API: Job, Mapper, Reducer, InputFormat, OutputFormat, Cluster, Partitioner
mapreduce/lib/input/	~25	FileInputFormat及子类: TextInputFormat, SequenceFileInputFormat, CombineFileInputFormat
mapreduce/lib/output/	~30	FileOutputFormat及子类: TextOutputFormat, SequenceFileOutputFormat, LazyOutputFormat
mapreduce/lib/map/	~10	Map阶段工具: InverseMapper, MultithreadedMapper, TokenCounterMapper
mapreduce/lib/reduce/	~5	Reduce阶段工具: IntSumReducer, LongSumReducer, WrappedReducer
mapreduce/lib/join/	~15	连接操作: CompositeInputFormat, TupleWritable
mapreduce/task/	~10	Context实现: MapContextImpl, ReduceContextImpl, TaskAttemptContextImpl
mapreduce/task/reduce/	~20	Shuffle核心: Shuffle, Fetcher, MergeManager, ShuffleScheduler, MapOutput
mapreduce/protocol/	~3	RPC协议: ClientProtocol, ClientProtocolProvider
mapreduce/security/	~8	安全组件: TokenCache, SecureShuffleUtils, IntermediateEncryptedStream
mapreduce/tools/	~3	CLI工具: CLI

子包	文件数	功能描述
mapreduce/lib/db/	~10	数据库IO: DBInputFormat, DBOutputFormat
mapreduce/lib/partition/	~5	分区策略: HashPartitioner, BinaryPartitioner, TotalOrderPartitioner
mapreduce/lib/aggregat e/	~15	聚合工具: ValueAggregator系列

4. 核心类关系分析

4.1 用户API层类关系

MapReduce 编程模型的核心抽象围绕 Mapper、Reducer、InputFormat、OutputFormat 四大组件展开。New API 采用抽象类而非接口，通过 Context 对象传递运行时环境，替代了 Old API 的回调模式。

类名	类型	核心方法	设计要点
Mapper	抽象类	setup() / map() / cleanup() / run()	模板方法模式: run()驱动setup→map 迭代→cleanup生命周期
Reducer	抽象类	setup() / reduce() / cleanup() / run()	同Mapper模式, 支持@Checkpointable注解标记可检查点
InputFormat	抽象类	getSplits() / createRecordReader()	策略模式: 分离分片计算与记录读取, 支持可插拔
OutputFormat	抽象类	getRecordWriter() / checkOutputSpecs() / getOutputCommitter()	工厂模式: 创建RecordWriter和OutputCommitter实例
Partitioner	抽象类	getPartition(key, value, numPartitions)	策略模式: 默认HashPartitioner, 控制key到reducer映射
RecordReader	抽象类	initialize() / nextKeyValue() / getCurrentKey() / getCurrentValue()	迭代器模式: 将InputSplit转换为key-value记录流
RecordWriter	抽象类	write(key, value) / close()	简洁接口: 只负责写入和关闭
Job	具体类	setMapper() / setReducer() / submit() / waitForCompletion()	Builder+Facade: 两阶段状态机(DEFINE→RUNNING), 封装配置和监控

4.2 任务执行层类层次

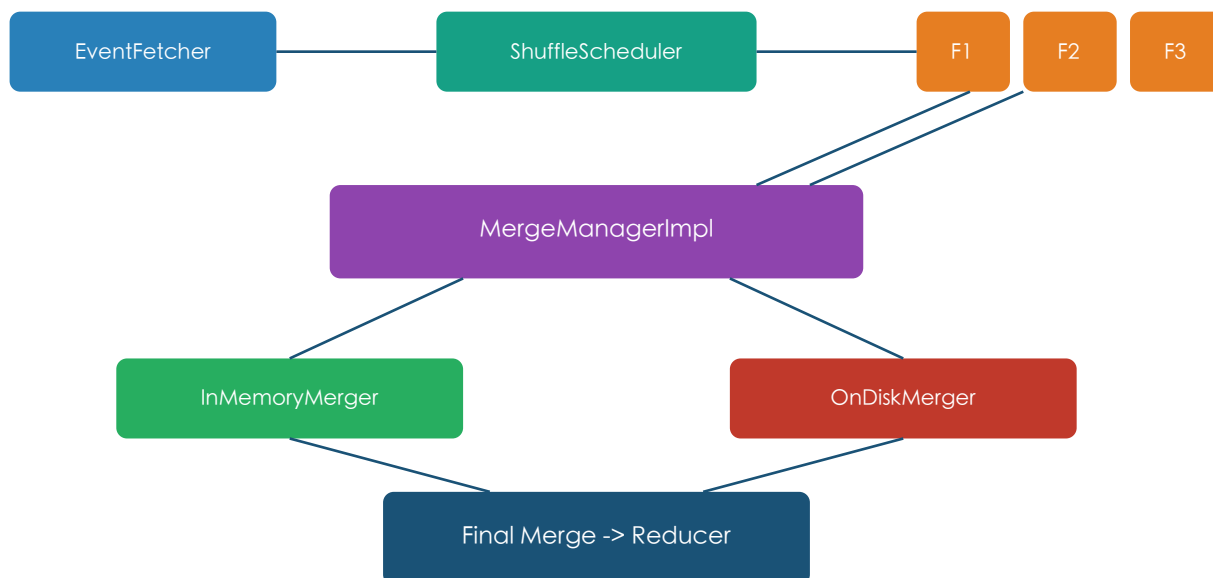
类名	父类	核心职责
Task (abstract)	-	任务基类: 管理TaskUmbilicalProtocol通信、进度汇报、Counters、加密密钥、Skip范围
MapTask	Task	Map任务: 反序列化InputSplit→运行Mapper→Sort/Spill→生成IFile分区输出
ReduceTask	Task	Reduce任务: Shuffle(copy→sort)→运行Reducer→输出结果

4.3 Shuffle组件关系

Shuffle 是 MapReduce 中最复杂的数据处理流程，由多个组件协同完成：

组件	类型	核心职责
Shuffle	入口协调者	创建并启动EventFetcher、Fetcher线程池、ShuffleScheduler, 协调整个shuffle流程直到完成
EventFetcher	事件监听线程	轮询TaskUmbilicalProtocol获取Map完成事件, 将Map输出位置信息传递给ShuffleScheduler
ShuffleSchedulerImpl	调度器	维护待拉取Map列表、管理失败主机惩罚延迟队列、控制并发拉取、监控Shuffle健康状态
Fetcher	拉取线程	从Map节点通过HTTP拉取Map输出数据, 支持SSL、重试、429限流处理
MergeManagerImpl	合并管理器	管理内存使用、触发内存→磁盘溢出、执行内存-内存/内存-磁盘/磁盘-磁盘多阶段合并
Merger.MergeQueue	归并引擎	基于优先队列的多路归并排序, 支持Segment抽象统一内存段和磁盘段
IFile.Writer/Reader	存储格式	Map中间输出的二进制存储格式, 支持压缩和校验和

Reduce端Shuffle数据流架构

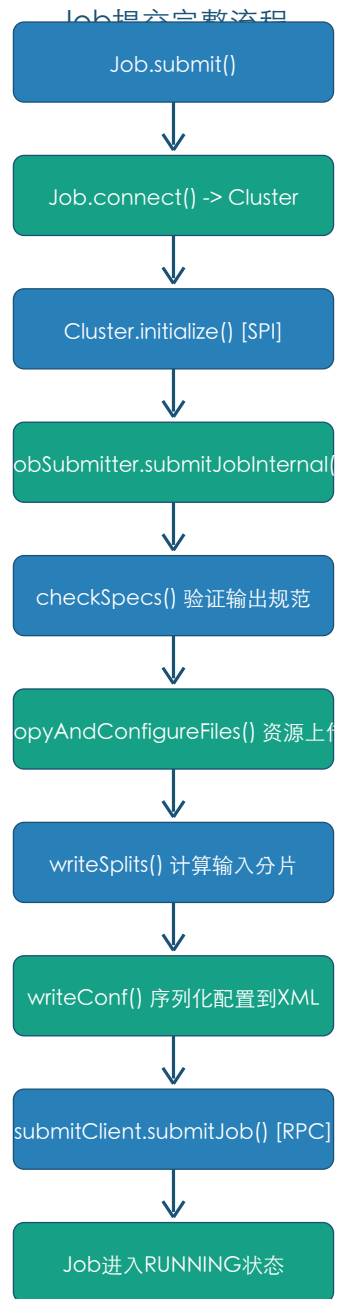


5. 设计模式分析

设计模式	应用位置	实现细节
模板方法模式 (Template Method)	Mapper.run() / Reducer.run()	run()定义固定算法骨架: setup→循环处理→cleanup; 子类只需重写map()/reduce()即可,无需关心生命周期控制
策略模式 (Strategy)	InputFormat / OutputFormat / Partitioner / OutputCommitter	可通过Job配置切换不同实现: TextInputFormat/SequenceFileInputFormat; HashPartitioner/TotalOrderPartitioner; FileOutputCommitter/ManifestCommitter
工厂模式 (Factory)	OutputFormat.getRecordWriter() OutputFormat.getOutputCommitter() PathOutputCommitterFactory	OutputFormat作为工厂同时创建RecordWriter和OutputCommitter; PathOutputCommitterFactory通过配置动态选择Committer实现
SPI服务发现 (Service Locator)	Cluster → ClientProtocolProvider	Cluster使用ServiceLoader发现协议提供者; 支持LocalClientProtocolProvider(本地模式)和YarnClientProtocolProvider(YARN模式)
两阶段提交 (Two-Phase Commit)	OutputCommitter	task commit: needsTaskCommit→commitTask(原子提升临时输出); job commit: commitJob(合并所有task输出到最终目录); 支持abortTask/abortJob回滚和recoverTask恢复
状态机模式 (State Machine)	Job.JobState(DDEFINE→ RUNNING)	Job对象有两个状态: DEFINE(配置态)和RUNNING(提交后); ensureState()守卫方法确保在正确状态下才能执行操作
观察者模式 (Observer)	EventFetcher → ShuffleScheduler	EventFetcher从TaskUmbilicalProtocol轮询事件, 通过resolve()回调通知ShuffleScheduler更新Map输出位置
生产者-消费者 (Producer-Consumer)	Fetcher → MergeManager → InMemoryMerger/OnDiskMerger	Fetcher生产Map输出数据; MergeManager管理内存缓冲; InMemoryMerger/OnDiskMerger作为后台消费者线程异步合并
Context对象模式 (Context Object)	MapContext / ReduceContext / TaskAttemptContext	New API使用Context对象传递运行时环境(Configuration+Counters+ OutputCommitter+RecordReader/Writer), 替代Old API的直接参数传递
装饰器模式 (Decorator)	FileOutputStream / FileInputStream	在标准输出/输入流上添加校验和计算功能, 透明地在数据写入/读取过程中维护数据完整性

6. 核心流程一：作业提交流程

作业提交是用户代码与集群交互的起点。从 `Job.submit()` 开始，经过一系列准备工作，最终通过 `RPC` 将作业提交到 `ResourceManager/JobTracker`。



6.1 关键步骤详解

Step 1: Cluster初始化 (SPI机制)

Cluster 使用 ServiceLoader<ClientProtocolProvider> 发现可用的协议提供者。遍历所有Provider尝试创建ClientProtocol实例，成功则返回。支持 LocalClientProtocolProvider (本地模式, mapreduce.framework.name=local) 和 YarnClientProtocolProvider (YARN模式, mapreduce.framework.name=yarn) 两种模式。采用双重检查锁定实现延迟初始化，避免重复创建。

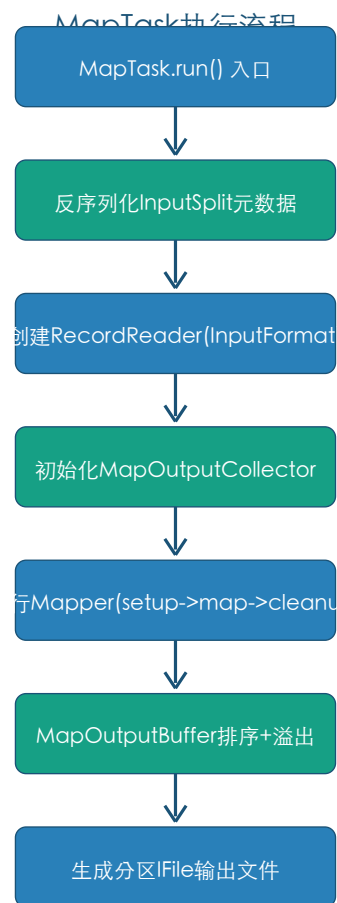
Step 2: JobSubmitter.submitJobInternal() 管道

步骤	方法	详细操作
1	checkSpecs()	调用OutputFormat.checkOutputSpecs()验证输出目录不存在
2	IP→HostName	将所有TaskTracker地址从IP转换为主机名,支持机架感知调度
3	分配JobID	通过ClientProtocol.getNewJobID()获取唯一作业标识
4	copyAndConfigureFiles()	上传作业JAR、依赖文件到HDFS暂存目录(staging area)
5	writeSplits()	调用InputFormat.getSplits()计算分片,序列化分片元数据到HDFS
6	Shuffle密钥生成	生成随机TokenIdentifier, 创建JobToken写入HDFS, 用于Shuffle安全验证
7	writeConf()	将完整JobConf序列化为XML写入HDFS暂存目录
8	submitJob()	通过ClientProtocol RPC提交作业, 作业进入RUNNING状态

Step 3: 输入分片算法 (FileInputFormat.getSplits)

分片大小计算公式: $splitSize = \max(\minSize, \min(maxSize, blockSize))$ 。其中 minSize 由 mapreduce.input.fileinputformat.split.minsize 控制 (默认1)，maxSize 由 mapreduce.input.fileinputformat.split.maxsize 控制 (默认Long.MAX_VALUE)。SPLIT_SLOP=1.1 容差阈值确保剩余文件不会因为略微超出splitSize就生成过小的分片。同时考虑数据本地性，将分片与HDFS block位置绑定，供调度器实现数据本地化调度。

7. 核心流程二: MapTask执行流程



7.1 进度分配模型

MapTask 将进度条按 2:1 分配给 Map 阶段和 Sort 阶段：当存在 Reduce 任务时, Map阶段占 66.7% (前2/3), Sort阶段占 33.3% (后1/3)。当不存在 Reduce 任务时, Map阶段占 100%。

7.2 MapOutputBuffer 排序收集器

MapOutputBuffer 是 Map 阶段的核心数据结构，它在一个环形缓冲区中收集 Mapper 输出，当缓冲区使用量达到阈值 (io.sort.spill.percent, 默认0.8) 时触发后台排序+溢出。排序使用快速排序算法，溢出时生成 IFile 格式的分区文件。所有溢出文件在 Map 阶段结束时通过 Merger 合并为最终的分区输出。如果配置了 Combiner, 排序后溢出前和最终合并时都会运行 Combiner 进行局部聚合。

关键配置: io.sort.mb (排序缓冲区大小, 默认100MB) | io.sort.factor (归并因子, 默认10) | io.sort.spill.percent (溢出阈值, 默认0.8) | mapreduce.map.combine.minspills (最少溢出文件数才运行Combiner, 默认3)

7.3 内部组件层次

组件	位置	职责
TrackedRecordReader	MapTask内部类	装饰RecordReader, 添加进度跟踪和字节数统计
SkippingRecordReader	MapTask内部类	支持跳过引起错误的记录(skip mode), 容错处理
MapOutputBuffer	MapTask内部类	环形缓冲区收集器, 负责排序、溢出、合并Map输出
NewOutputCollector	MapTask内部类	适配New API Mapper输出到MapOutputBuffer
NewTrackingRecordReader	MapTask内部类	适配New API RecordReader并添加计数器

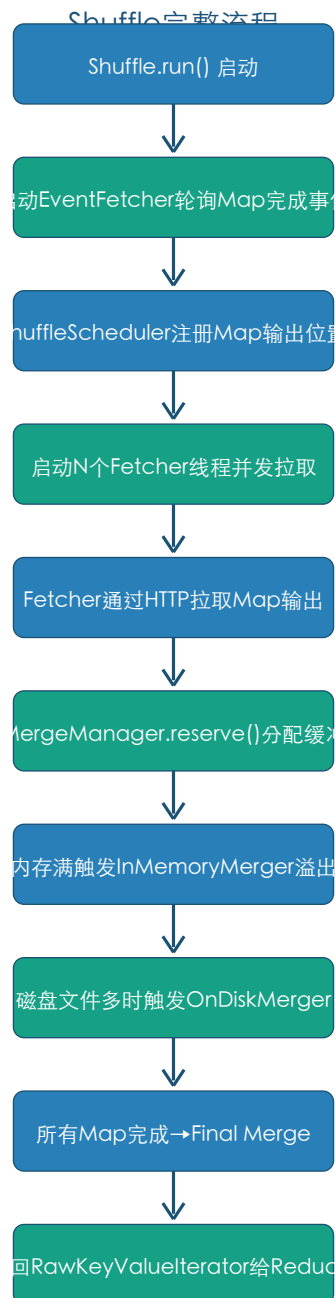
8. 核心流程三: ReduceTask与Shuffle流程

ReduceTask 的执行分为三个明确的阶段: Copy (Shuffle)、Sort (Merge)、Reduce。其中 Shuffle 是最复杂的阶段，涉及多线程并发拉取和多级合并。

8.1 ReduceTask 三阶段

阶段	进度占比	核心操作	关键组件
Copy (Shuffle)	33%	从Map节点拉取属于本Reduce的分区数据	Shuffle, Fetcher, ShuffleScheduler
Sort (Merge)	33%	将所有Map输出合并为全局有序的键值流	MergeManagerImpl, Merger, IFile
Reduce	34%	迭代排序后的键值对执行Reducer逻辑	Reducer, ReduceContextImpl

8.2 Shuffle详细流程



8.3 MergeManager 内存管理策略

MergeManagerImpl 精心管理内存使用，避免OOM同时最大化性能：

参数	默认值	作用
shuffle.input.buffer.percent	0.70	Reduce堆内存用于Shuffle缓冲的比例
shuffle.memory.limit.percent	0.25	单个Map输出占Shuffle缓冲的最大比例, 超过则直接写磁盘
shuffle.merge.percent	0.66	已提交内存达此比例触发InMemoryMerger溢出

参数	默认值	作用
io.sort.factor	10	OnDiskMerger单次合并的最大文件数
reduce.memtomem.enabled	false	是否启用内存-内存合并优化

8.4 Fetcher拉取机制

每个 Fetcher 线程工作循环: 1) 等待MergeManager资源可用 → 2) 从ShuffleScheduler获取一个待拉取的MapHost → 3) 通过HTTP GET从该主机批量拉取多个Map输出 → 4) 每个Map输出按大小决定存储位置(内存或磁盘) → 5) 提交给MergeManager。支持 SSL 加密传输、429 限流退避、连接重试、以及 Shuffle 安全令牌 HMAC 验证。失败处理采用指数退避惩罚策略: $penalty = 10s \times 1.3^{failures}$ ，最大不超过配置上限。

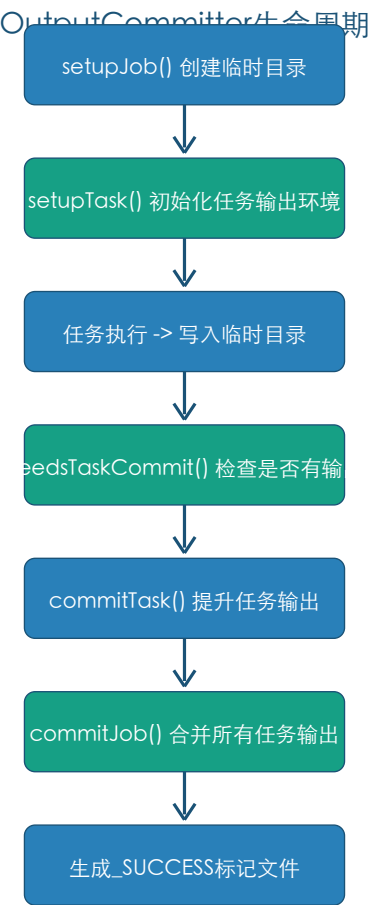
8.5 ReduceContextImpl 值迭代器

ReduceContextImpl 是 Reduce 阶段的核心上下文实现，它负责将排序后的键值流按 key 分组提供给 Reducer。内部 ValueIterator 使用 RawComparator 判断相邻记录是否属于同一个 key group。支持 mark()/reset() 机制，当值迭代器需要重复扫描时，通过 BackupStore 将已读取的值缓存到内存/磁盘，支持回退重读。

9. 核心流程四: 输出提交协议

OutputCommitter 定义了 MapReduce 的两阶段提交协议，确保在推测执行、任务失败重试等场景下输出的原子性和一致性。

9.1 提交协议生命周期



9.2 FileOutputCommitter 双版本算法

特性	算法版本1	算法版本2 (默认)
commitTask行为	将任务输出移到作业临时目录下的task子目录	直接将任务输出移到最终输出目录
commitJob行为	将所有task子目录合并到 最终输出目录	仅删除临时目录并生成_SUCCESS标记文件
原子性	作业级原子: 要么全部可见, 要么全不可见	任务级原子: 每个task输出 单独可见
性能	commitJob慢(大量文件rename)	commitJob快(O(1)操作)

特性	算法版本1	算法版本2 (默认)
推测执行	安全: 仅commit获胜者	安全: rename原子保证
适用场景	需要作业级原子性的场景	大规模作业, 对象存储

9.3 临时目录结构

FileOutputCommitter 使用以下目录层次管理临时输出：

```
output/_temporary/ # PENDING_DIR_NAME
output/_temporary/{appAttemptId}/ # 
output/_temporary/{appAttemptId}/_temporary/ # 
output/_temporary/{appAttemptId}/_temporary/{taskAttemptId}/ # 
output/_temporary/{appAttemptId}/{taskId}/ # (V1)
```

10. 时序图: Job提交与执行

以下展示从用户调用 `Job.waitForCompletion()` 到作业完成的关键交互序列：

```
UserCode Job Cluster JobSubmitter ClientProtocol(RM)
| | | | | | |
|--submit()----->| | | |
| |--connect()-->| | |
| | |--SPI load--->| |
| | |<--provider---| |
| |--submitJob()----->| |
| | | |--checkSpecs() |
| | | |--writeSplits() |
| | | |--writeConf() |
| | | |--submitJob()-->|
| | | |<--jobId-----|
| | | |<--RUNNING----| | |
|--waitFor----->| | | |
| |--poll status----->|----getStatus-->|
| | | |<--JobStatus----|
|<--completed-----| | | |
```

11. 时序图: Shuffle数据拉取

```
ReduceTask Shuffle EventFetcher ShuffleScheduler Fetcher MergeManager
| | | | | |
|--run()----->| | | |
| |--start()----->| | |
| |--start(N)-----|----->| (N threads) |
| | | | |
| | |--getMapEvents-->| |
| | | (from Umbilical)| |
| | |--resolve()----->| |
| | | |--getHost()-->| |
| | | | |--HTTP GET--->|
| | | | | (MapOutput) |
| | | | |--reserve()-->|
| | | | |<-InMemory----|
| | | | |--commit()----->|
| | | | |
| | | | (threshold reached) |
| | | | |--merge()----->|
| | | | | (spill to disk)|
| | | | |
| |<--finalMerge()-----| |
|<-iterator----| | | |
```

12. 关键场景调用链分析

12.1 场景: FileInputFormat 分片计算

```
FileInputFormat.getSplits(job)
→ listStatus(job) // 
→ TokenCache.obtainTokensForNamenodes() // 
→ singleThreadedListStatus() / LocatedFileStatusFetcher ( )
→ :
→ computeSplitSize(blockSize, minSize, maxSize) // max(min, min(max, block))
→ isSplittable() 
→ makeSplit(file, start, length, hosts) // FileSplit
→ SPLIT_SLOP=1.1 : < splitSize*1.1
```

12.2 场景: MapTask 排序溢出

```
Mapper.map(key, value, context)
→ context.write(key, value)
→ MapOutputBuffer.collect(key, value, partition)
→ (kvbuffer + kvmeta)
→ spill → startSpill()
→ SpillThread.sortAndSpill()
→ QuickSort
→ [ ] Combiner
→ IFile.Writer
→ Map → mergeParts()
→ Merger.merge() 
→ file.out + file.out.index
```

12.3 场景: Shuffle失败重试与惩罚

```
Fetcher.copyFromHost(host)
→ ShuffleScheduler.copyFailed(mapId, host, readError, connectExcp)
→ failureCounts[mapId]++ // Map
→ hostFailures[hostname]++ // 
→ failures >= abortFailureLimit(max(30, totalMaps/10))
→ reportExceptionReduce
→ checkAndInformMRAppMaster() // AM
→ checkReducerHealth() // 
→ penalize(host, min(10s × 1.3^failures, maxPenalty))
→ DelayQueue 
→ Referee.pendingHosts
```

12.4 场景: FileOutputCommitter V2 提交流程

```
→ commitTask(context)
→ getTaskAttemptPath() // output/_temporary/{appAttempt}/_temporary/{taskAttempt}/
→ getOutputPath() // output/ ( )
→ mergePaths(fs, taskAttemptPath, outputPath) // 
→ [ ] cleanup task temporary directory
```

```

■■■■■■■■ → commitJob(context)
→ cleanupJob(context) // V2: ■■■_temporary■■
→ fs.delete(getPendingJobAttemptsPath(), true)
→ [■■skipCleanup=false] ■■_temporary
→ createSuccessMarker() // ■■_SUCCESS■■■■

```

12.5 场景: IFile写入与合并

```

IFile.Writer.append(key, value)
→ keySerializer.serialize(key) // ■■■key■buffer
→ valueSerializer.serialize(value) // ■■■value■buffer
→ WritableUtils.writeVInt(out, keyLength) // ■key■■(VInt■■)
→ WritableUtils.writeVInt(out, valueLength) // ■value■■
→ out.write(buffer) // ■■■■■■
→ [■■■■] FSDataOutputStream → IFileOutputStream(checksum) → CompressionOutputStream

Writer.close()
→ writeVInt(EOF_MARKER=-1) × 2 // ■EOF■■
→ compressedOut.finish() // ■■■■
→ checksumOut.finish() // ■■■■
→ CodecPool.returnCompressor() // ■■■■

```

12.6 场景: Merger多路归并排序

```

Merger.merge(segments, mergeFactor)
→ MergeQueue extends PriorityQueue // ■■■■■■
→ ■segment■■■■■(■■■■)
→ ■■■■■■:
→ ■■■■: ■■■N■segments (N■■■■■■■■mergeFactor■)
→ ■■■■: ■■■■mergeFactor■segments■■■■■
→ ■■■■: ■■■■■■■■(■■■■, ■■■■■■■■)
→ ■■■■: put(segment) → adjustTop() → getKey()/getValue()
→ ■■■■ vs ■■■■■■■■Segment■■, Reader■■■■■■■■

```

13. 配置参数速查

13.1 输入/分片配置

参数名	默认值	说明
mapreduce.input.fileinputformat.split.minsize	1	分片最小字节数
mapreduce.input.fileinputformat.split.maxsize	Long.MAX	分片最大字节数
mapreduce.input.fileinputformat.input.dir.recursive	false	是否递归遍历输入目录
mapreduce.input.fileinputformat.list-status.num-threads	1	列取文件状态的并发线程数

13.2 Map端配置

参数名	默认值	说明
mapreduce.task.io.sort.mb	100	Map排序缓冲区大小(MB)
mapreduce.task.io.sort.factor	10	归并因子(每次最多合并文件数)
mapreduce.map.sort.spill.percent	0.80	缓冲区使用率达此比例触发溢出
mapreduce.map.combine.minspills	3	运行Combiner所需最少溢出文件数

13.3 Shuffle/Reduce端配置

参数名	默认值	说明
mapreduce.reduce.shuffle.input.buffer.percent	0.70	Reduce堆用于Shuffle的比例
mapreduce.reduce.shuffle.memory.limit.percent	0.25	单个Map输出的最大内存占比
mapreduce.reduce.shuffle.merge.percent	0.66	触发内存合并溢出的阈值
mapreduce.reduce.shuffle.parallelcopies	5	并发拉取线程数
mapreduce.reduce.shuffle.connect.timeout	180000	Shuffle连接超时(ms)
mapreduce.reduce.shuffle.read.timeout	180000	Shuffle读取超时(ms)

13.4 输出提交配置

参数名	默认值	说明
mapreduce.fileoutputcommitter.algorithm.version	2	提交算法版本(1或2)
mapreduce.fileoutputcommitter.cleanup.skipped	false	是否跳过临时目录清理
mapreduce.fileoutputcommitter.cleanup-failures.ignored	false	是否忽略清理失败异常
mapreduce.fileoutputcommitter.failures.attempts	1	作业提交失败重试次数
mapreduce.fileoutputcommitter.task.cleanup.enabled	false	V2: 任务完成后是否删除临时目录

14. 总结

hadoop-mapreduce-client-core 作为 Hadoop MapReduce 的客户端核心模块，以 537 个 Java 源文件、五层清晰架构实现了完整的分布式数据处理管道。

14.1 架构亮点

特性	说明
双API兼容	通过Old API/New API双轨并行,保证了向后兼容性。New API使用Context对象模式替代回调,更加面向对象,但底层共享任务执行引擎(MapTask/ReduceTask)。
SPI可扩展	Cluster通过ServiceLoader发现ClientProtocolProvider,使得同一份客户端代码可以透明运行在本地模式和YARN模式上,体现了开闭原则。
Shuffle精细化管理	MergeManagerImpl通过多级阈值控制内存使用,InMemoryMerger和OnDiskMerger作为后台线程异步合并,Fetcher支持SSL/限流/指数退避,实现了高效可靠的数据传输。
两阶段提交	OutputCommitter的task commit + job commit两阶段协议,配合V1/V2双版本算法,在原子性和性能之间提供了灵活选择,V2对对象存储场景做了专门优化。
多路归并排序	Merger.MergeQueue基于优先队列实现 $O(N \cdot \log K)$ 多路归并,统一了内存段和磁盘段的抽象,分轮次合并策略确保内存可控同时最小化IO次数。

14.2 核心数据流总结

完整数据流：	<code>InputFormat.getSplits()</code>	→	<code>RecordReader</code>	→	<code>Mapper.map()</code>	→	<code>MapOutputBuffer (sort+spill)</code>	→	<code>IFile</code>	→	[Shuffle: <code>Fetcher</code> → <code>MergeManager</code> → <code>Merger</code>]	→	<code>ReduceContextImpl.ValueIterator</code>	→	<code>Reducer.reduce()</code>	→	<code>RecordWriter</code>	→	<code>OutputCommitter.commitTask()/commitJob()</code>
--------	--------------------------------------	---	---------------------------	---	---------------------------	---	---	---	--------------------	---	--	---	--	---	-------------------------------	---	---------------------------	---	---

14.3 模块统计

指标	数值
Java源文件总数	537
Old API (mapred) 文件数	205
New API (mapreduce) 文件数	332
核心类	~30 (Mapper/Reducer/Job/Task/MapTask/ReduceTask/Shuffle/Merger等)
设计模式	10种 (模板方法/策略/工厂/SPI/两阶段提交/状态机/观察者/生产者消费者/Context/装饰器)

指标	数值
Git分支	release-3.3.5-RC0